



**FACULTY OF COMPUTER SCIENCE AND MATHEMATICS
BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH
HONOURS**

**CSE3433 SOFTWARE ARCHITECTURE
SEMESTER II SESSION 25/26**

**PROJECT TITLE:
INDIVIDUAL LOGBOOK**

**TALENTFLOW – EVENT-DRIVEN JOB RECRUITMENT & APPLICATION
PROCESSING SYSTEM**

PREPARED FOR:

Dr. Mohamad Nor Bin Hassan

PREPARED BY:

NAME	MATRIC NO
NURUL IFFAH SABRINA BINTI AHMAD SHUKRI	S75894
IZZ DHIA DAMIA BINTI MAT ZAIDI	S75849

Team Information

Field	Details
My Name	Nurul Iffah Sabrina Binti Ahmad Shukri (S75894)
My Role	System Architect & Designer
Teammate	Izz Dhia Damia Binti Mat Zaidi (S75849) – Backend & Integration Lead

Weekly Progress Log

Week 4: Project Planning & Research

Undertaken Tasks:

- Conducted research on Event-Driven Architecture (EDA) concepts and patterns
- Studied job recruitment system requirements and existing solutions
- Defined project scope, objectives, and title (TalentFlow)
- Identified system stakeholders: candidates and recruiters
- Discussed team responsibilities with Izz Dhia Damia and agreed on EDA as the guiding architecture style

Individual Contributions:

- Analyzed existing recruitment systems to identify gaps and inefficiencies
- Documented functional requirements such as job posting, resume screening, and notifications
- Drafted the initial problem statement describing manual HR bottlenecks and poor candidate communication
- Proposed EDA with publish-subscribe model as the architecture style

Challenges:

- Understanding the difference between synchronous and asynchronous communication in EDA
- Defining clear service boundaries for the recruitment domain

Action Plans:

- Review more EDA examples from online resources and academic references
- Discuss service boundary definitions with teammate Izz Dhia Damia
- Finalize event list and system objective for Deliverable 1 submission

Week 5: Architecture Design

Undertaken Tasks:

- Designed overall TalentFlow system architecture using EDA
- Identified core services: Job, Application, Resume Filter, Status Management, Interview, Notification, Dashboard
- Defined event flow: ApplicationSubmitted → ResumeScreened → StatusUpdated → Notified
- Created initial architecture diagrams (System Context Diagram)
- Selected technology stack: Node.js, Postman, VSCode

Individual Contributions:

- Designed the System Context Diagram (Figure 1) showing interactions between Candidate, Recruiter, and external systems
- Defined service responsibilities and events produced/consumed for each service
- Documented architecture style justification for EDA with publish-subscribe pattern
- Created the component-responsibility table included in the report

Challenges:

- Deciding between event choreography and orchestration for the system flow
- Determining which service owns which data in an event-driven setup

Action Plans:

- Finalize all four architecture diagrams for Deliverable 2
- Document event contracts clearly for each service
- Coordinate with teammate for backend service planning

Week 6: Environment Setup

Undertaken Tasks:

- Set up development environment using VSCode
- Installed Node.js and verified npm package manager
- Set up Postman for API testing
- Reviewed the project folder structure from the zip file
- Confirmed tools to be used: Node.js, Postman, VSCode (no Docker/Redis for this prototype)

Individual Contributions:

- Installed and configured VSCode with relevant extensions (ESLint, REST Client)
- Set up Postman workspace and created initial request collections
- Verified Node.js version and initialized npm project
- Assisted in setting up the basic project folder structure

Challenges:

- Understanding which parts of the environment setup were needed for the simplified prototype
- Ensuring consistent Node.js version across the development machine

Action Plans:

- Create a simple setup guide document for the project
- Test basic Node.js server startup
- Verify Postman can call local API endpoints

Week 7: Core Service Implementation (Observation Week)

Undertaken Tasks:

- Began implementation of Application Service (event producer)
- Reviewed project code structure from recruitment-eda-system11
- Understood how events flow between services in the codebase
- Verified Postman can call the Application Service endpoints
- Prepared for Project Observation 1

Individual Contributions:

- Reviewed and understood the Application Service routes and event publishing logic
- Tested POST /applications endpoint using Postman
- Documented API endpoint structure for the Application Service
- Confirmed application submission triggers the correct event

Challenges:

- Understanding the internal event bus implementation without a real message broker
- Ensuring request validation returns proper error messages

Action Plans:

- Test more edge cases via Postman for the Application Service
- Document the event payloads used in the system
- Review Screening Service implementation next

Week 8: Screening & Status Services**Undertaken Tasks:**

- Reviewed the Screening/Resume Filter Service in the codebase
- Traced how ApplicationSubmitted event triggers resume screening
- Understood keyword-based scoring logic used in the prototype
- Reviewed Status Management service and event handling
- Tested screening flow using Postman

Individual Contributions:

- Verified that Resume Filter correctly consumes ApplicationSubmitted events
- Traced the ResumeScreened event and confirmed Status Manager updates accordingly
- Documented the scoring algorithm logic used for resume filtering
- Tested candidate status update flow via Postman

Challenges:

- Understanding the scoring logic and how keywords are matched
- Tracing async event flows without a visual debugger

Action Plans:

- Add more test cases with different skill keywords
- Document screening result model and status values
- Review Interview Service next week

Week 9: Interview & Notification Services (Observation Week)

Undertaken Tasks:

- Reviewed Interview Service and scheduling logic
- Traced how shortlisted status triggers interview scheduling
- Reviewed Notification Service and how it sends confirmation messages
- Tested end-to-end flow: apply → screen → shortlist → interview → notify
- Prepared for Project Observation 2

Individual Contributions:

- Verified Interview Service consumes StatusUpdated (shortlisted) event correctly
- Confirmed Notification Service sends simulated email on InterviewScheduled event
- Tested complete end-to-end flow from application submission to notification via Postman
- Documented notification event payloads

Challenges:

- Ensuring the interview scheduling logic handles the correct status values
- Verifying notification is triggered only under correct conditions

Action Plans:

- Finalize all service integration testing
- Prepare documentation for the full event flow
- Begin writing report sections

Week 10: Integration & End-to-End Testing

Undertaken Tasks:

- Conducted full end-to-end integration testing of all services
- Created Postman test collections covering the complete recruitment workflow
- Identified and noted minor issues in event handling
- Validated that the system flow matches the architecture design
- Verified all events are correctly published and consumed

Individual Contributions:

- Created organized Postman collection for testing all API endpoints
- Ran end-to-end tests: job posting → application → screening → interview → notification
- Confirmed system behaviour matches the EDA event flow in the architecture design
- Documented test results and noted edge cases observed

Challenges:

- Tracing async event chains across multiple services

- Ensuring consistent test data across all service tests

Action Plans:

- Document test results for the prototype section of the report
- Begin writing the report's Prototype Implementation section
- Clean up Postman collection for submission evidence

Week 11: Report Writing – Architecture Sections

Undertaken Tasks:

- Wrote report sections: Introduction, System Overview, Problem Description
- Wrote System Objectives and Selected Architecture Style sections
- Wrote Functional Requirements and Non-Functional Requirements tables
- Wrote Architecture Style Explanation and Service/Component Identification table
- Wrote Team Member Roles section

Individual Contributions:

- Authored the Introduction and System Overview sections in full
- Wrote the Problem Description highlighting HR inefficiencies
- Wrote the System Objectives section with 6 clear objectives
- Prepared the Functional Requirements (FR01–FR07) and Non-Functional Requirements tables
- Wrote Architecture Style Explanation describing EDA and publish-subscribe model
- Wrote the Service/Component Identification table with events produced and consumed
- Wrote the Team Member Roles section

Challenges:

- Writing clearly and concisely about technical concepts for an academic report
- Coordinating which sections each team member would write

Action Plans:

- Review written sections with teammate
- Finalize Deployment Diagram explanation
- Submit final report by Week 13 deadline

Week 12: Report Writing – Diagrams & Finalisation

Undertaken Tasks:

- Finalised and explained the System Context Diagram (Figure 1) in the report
- Wrote explanation for Deployment Diagram (Figure 1.4)
- Reviewed and proofread full report for completeness
- Coordinated with teammate on Interaction Diagram and Component Diagram sections
- Prepared GitHub repository and README documentation

Individual Contributions:

- Wrote the explanation for the System Context Diagram (Figure 1)
- Wrote the explanation for the Deployment Diagram (Figure 1.4)
- Proofread full report and fixed grammar/formatting issues
- Reviewed Table of Contents and ensured all sections were present

Challenges:

- Writing accurate explanations for diagrams created collaboratively
- Ensuring the report sections align and flow logically

Action Plans:

- Finalise report and submit by Week 13 deadline
- Prepare individual logbook entries
- Prepare demonstration materials for Week 14

GenAI Usage Record

The following table documents all use of Generative AI tools throughout the project.

Week	Tool	Purpose	Prompt Used	Output Summary	How It Was Used
Week 4	ChatGPT	Understand EDA concepts	Explain Event-Driven Architecture with examples in software systems	Overview of EDA, publish-subscribe model, and use cases	Used to build foundational understanding before starting design
Week 5	ChatGPT	Architecture justification writing	How to justify Event-Driven Architecture for a recruitment system	Key points on async processing, loose coupling, scalability	Adapted into the Architecture Style Explanation section of the report
Week 6	Deepseek	Node.js project setup	How to set up a basic Node.js project structure for an event-driven system	Folder structure, package.json setup, basic express server	Used as reference for understanding the project structure
Week 8	ChatGPT	Understand scoring logic	How to implement a simple keyword-based resume scoring in Node.js	Example scoring function with keyword arrays	Helped understand the screening algorithm in the codebase
Week 10	Claude	Postman testing tips	How to test an event-driven REST API using Postman	Steps to organise Postman collections and run sequential tests	Followed the guidance to organise and run integration tests

Week 11	Gemini	Report writing assistance	Help me write a system overview for a recruitment automation system using EDA	Draft paragraph on EDA benefits for recruitment	Used as a starting point, then rewritten in own words for the report
Week 12	ChatGPT	Proofreading guidance	What are common mistakes in software architecture academic reports	Checklist of common issues: vague requirements, missing justification, weak diagrams	Used as a checklist to review and improve the final report

Reflection

My Role in the Project

As the System Architect & Designer, my primary responsibility was to design the overall system structure, create the architecture diagrams, write the key report sections, and ensure the design decisions were clearly justified and documented. We both build the backend services and understood the codebase, tested all API endpoints using Postman, and validated that the implementation matches our architecture design.

Key Learnings

- **Event-Driven Architecture:** I now understand how asynchronous event flow enables loose coupling between services. Events like `ApplicationSubmitted` and `ResumeScreened` replace direct service-to-service calls.
- **Postman for API Testing:** I learned how to organise and run sequential API tests to verify end-to-end event flows without needing a frontend interface.
- **Architecture Documentation:** Writing clear architecture diagrams and justification taught me how design decisions must be traceable from requirements to implementation.
- **Report Writing:** I developed the ability to explain complex technical systems in structured academic writing across multiple sections.

Challenges Overcome

- **Understanding Async Flows:** Without a visual debugger, tracing event chains across services was difficult. I overcame this by carefully reading the code and testing each step individually in Postman.
- **Distinguishing Architecture Roles:** Initially I was unsure which diagrams I was responsible for. After discussing with my teammate, I took ownership of the System Context and Deployment Diagram explanations.
- **Time Management:** Balancing architecture design work, report writing, and testing across multiple weeks required careful planning of my weekly tasks.

Future Improvements

- **Add a real message broker** (RabbitMQ or Kafka) to replace the simulated event bus for more realistic EDA behaviour.
- **Implement a simple frontend interface** so users can submit applications without needing Postman.
- **Add monitoring and logging tools** to visualise event flow in real time.
- **Extend the resume filter** with NLP-based keyword extraction for smarter candidate scoring.